

GOMGOM-AI 화면-코드-데이터 흐름 인수인계 문서

목표: 처음 맡는 개발자가 화면 입력, 서버 처리, Database(DB) 저장, 결과 출력의 연결 순서를 실제 코드 기준으로 빠르게 이해하도록 정리했다.

용어: GPT(Generative Pre-trained Transformer)·GPS(Global Positioning System)·JSON(JavaScript Object Notation)·DB(Database).

검증 범위: 프론트엔드 템플릿/스크립트, Django 뷰와 보조 모듈, Recommendation 저장 스키마, 관리자 등록 여부.

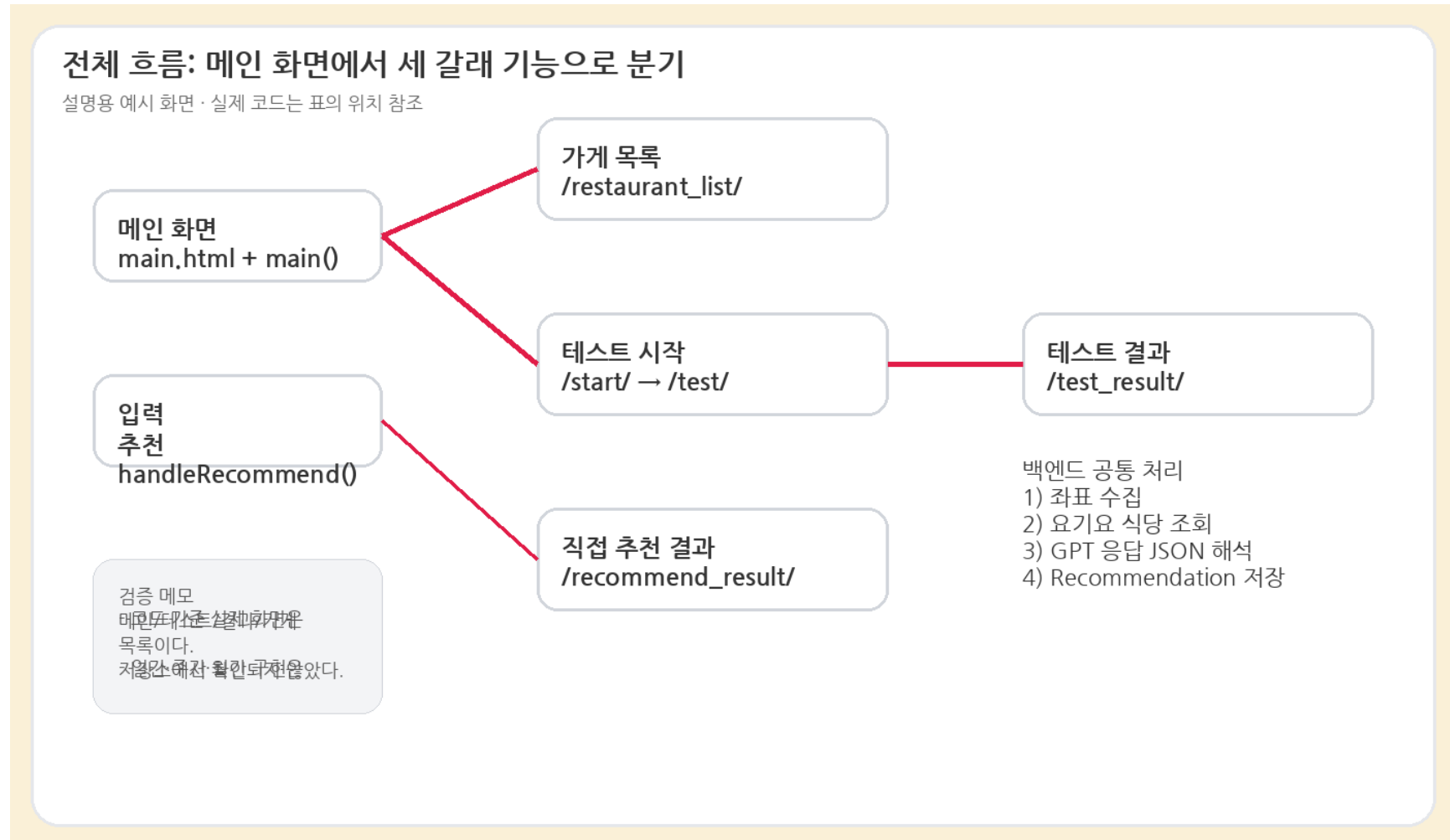
검증 제외: 작업 공간에서 일간·주간·월간 구현 또는 별도 첨부 캡처 파일을 확인하지 못해, 해당 명칭의 기능은 추측으로 채우지 않고 제외했다.

아래 페이지부터는 화면 그룹별로 입력 → 처리 → 출력 → 저장 흐름을 연속 페이지로 묶어 설명한다.

전체 흐름: 메인 화면에서 세 갈래 기능으로 분기되는 구조

현재 설명 기능과 계층: 전체 > Frontend 화면 분기 > Backend 라우트 > 외부 API > DB 저장

모든 사용자 흐름은 / 에서 시작한다 · 좌표가 있으면 가게 목록 또는 추천 결과로 이동한다 · 추천 기능은 공통적으로 요기요 목록 조회와 GPT 응답 해석을 거친다



예시 값과 재구성 화면은 설명용이며, 실제 연결 판단은 다음 페이지 표의 코드 위치를 기준으로 했다.

전체 흐름: 메인 화면에서 세 갈래 기능으로 분기되는 구조 · 설명 표

현재 설명 기능과 계층: 전체 > Frontend 화면 분기 > Backend 라우트 > 외부 API > DB 저장

번호	화면 요소와 쉬운 역할	입력·처리·출력	실제 코드 위치
1	첫 진입 화면	입력 없음 → main()이 main.html을 렌더링 → 사용자가 세 갈래 기능 중 하나를 선택한다.	urls.py:14, views.py:165-166, templates/gomgom_ai/main.html:20-90
2	가게 목록 빠른 이동	위치 좌표(lat, lng)를 모으면 /restaurant_list/로 이동한다. 이 경로는 추천 저장 없이 조회만 수행한다.	main.html:30-31, 125-130 / views.py:291-325
3	테스트 기반 추천 시작	text, lat, lng를 /start/ 와 /test/ 로 넘겨 질문형 추천을 시작한다.	main.html:156-159, 172-190 / views.py:174-183, 157-158
4	직접 추천 결과	자유 입력 text 와 좌표를 /recommend_result/로 보내 분류형 추천을 실행한다.	main.html:161-168 / views.py:511-625
5	테스트 결과 저장	type1~type6를 집계해 score를 만들고 Recommendation에 저장한 뒤 결과 화면을 보여준다.	test.html:91-126 / views.py:391-507

참고: static/javascript 폴더에 여러 실험성 스크립트가 있으나, 실제 템플릿에서 연결이 확인된 파일은 main.js이고 테스트 질문 흐름은 test.html 내부 스크립트가 기준이다.

메인 화면: 자유 입력, 위치 수집, 기능 분기가 한곳에서 시작된다

현재 설명 기능과 계층: 메인 화면 > Template(main.html) > main.js > Inline JavaScript

사용자는 자유 입력 또는 바로 테스트를 선택한다 · 브라우저가 GPS 좌표를 받으면 다음 화면 URL에 붙인다 · 위치 조회 실패 시 기본 좌표 37.484934, 126.981321로 대체한다



예시 값과 재구성 화면은 설명용이며, 실제 연결 판단은 다음 페이지 표의 코드 위치를 기준으로 했다.

메인 화면: 자유 입력, 위치 수집, 기능 분기가 한곳에서 시작된다 · 설명 표

현재 설명 기능과 계층: 메인 화면 > Template(main.html) > main.js > Inline JavaScript

번호	화면 요소와 쉬운 역할	입력·처리·출력	실제 코드 위치
1	상단 로고 링크	클릭 입력 없음 → 메인으로 되돌아가는 홈 링크 역할 → 출력은 같은 / 경로 재진입이다.	main.html:20-27, urls.py:14
2	내 주변 가게 버튼	입력값은 latestLat, latestLng → waitForGeolocation()이 좌표 준비를 기다림 → /restaurant_list/?lat=...&lng;=... 로 이동한다.	main.html:30-31, 125-147
3	말풍선 안내 문구	사용자 입력 없음 → main.js typing()이 문자열을 한 글자씩 표시 → 첫 진 입 안내를 담당한다.	static/javascript/main.js:1-19, main.html:38-41
4	자유 입력창	입력 이름은 #userInput → 사용 의미는 먹고 싶은 음식·기분·상황 텍스트 → handleRecommend()와 테스트 버튼 흐름이 공통으로 읽는다.	main.html:55-63, 149-168, 172-190
5	입력 추천 버튼	입력은 text + 좌표 → 먼저 테스트 병행 여부 모달을 띄움 → 아니요면 /recommend_result/, 네면 /start/ 로 분기한다.	main.html:62, 149-168
6	입맛 테스트 버튼	입력은 선택적 text + 좌표 → text가 비어 있으면 emptyInputModal 확인 후 /start/로 이동 → 결과적으로 테스트형 흐름을 시작한다.	main.html:65, 82-89, 172-192

테스트 시작 화면: 이전 입력을 유지하고 질문 화면으로 넘긴다

현재 설명 기능과 계층: 테스트 그룹 1/2 > start.html > start_view()

이 화면은 계산을 하지 않고 전달만 담당한다 · 메인 화면에서 받은 text, lat, lng를 그대로 보여주고 다음 단계로 넘긴다



예시 값과 재구성 화면은 설명용이며, 실제 연결 판단은 다음 페이지 표의 코드 위치를 기준으로 했다.

테스트 시작 화면: 이전 입력을 유지하고 질문 화면으로 넘긴다 · 설명 표

현재 설명 기능과 계층: 테스트 그룹 1/2 > start.html > start_view()

번호	화면 요소와 쉬운 역할	입력·처리·출력	실제 코드 위치
1	입력값 확인 문장	입력은 query string의 text → start_view()가 컨텍스트로 전달 → 화면에 사용자가 방금 넣은 문장을 재확인한다.	views.py:174-183, start.html:29-32
2	다음으로 버튼	추가 입력 없음 → goNextQuestion()가 기존 text, lat, lng를 읽어 /test/ URL을 만든다 → 질문 화면으로 이동한다.	start.html:32, 59-69
3	전달 유지 안내	lat/lng가 없으면 콘솔에 경고만 남기고 값은 그대로 둔다 → 이 단계에서는 저장이나 외부 호출이 없다.	start.html:43-57

질문 화면: 6개 답변이 태그 배열이 되어 결과 화면 입력으로 바뀐다

현재 설명 기능과 계층: 테스트 그룹 2/2 > test.html > Inline JavaScript

질문과 답변은 test.html 내부 qnaList에 하드코딩되어 있다 · 각 버튼은 화면 문장과 별개로 active, calm, spicy 같은 type 값을 가진다

질문 화면: 6개 답변을 태그 목록으로 누적

설명용 예시 화면 · 실제 코드는 표의 위치 참조

1

Q1. 혼밥을 하러 갔을 때, 어떤 분위기의 식당을 더 좋아하나요?

a. 사람 많고
북적북적한 식당

2

b. 조용하고
아늑한 식당

3

선택할 때마다 active/calm 같은 type 값이 selectedTypes 배열에 쌓인다.

4

예시 값과 재구성 화면은 설명용이며, 실제 연결 판단은 다음 페이지 표의 코드 위치를 기준으로 했다.

질문 화면: 6개 답변이 태그 배열이 되어 결과 화면 입력으로 바뀐다 · 설명 표

현재 설명 기능과 계층: 테스트 그룹 2/2 > test.html > Inline JavaScript

번호	화면 요소와 쉬운 역할	입력·처리·출력	실제 코드 위치
1	질문 박스	입력은 current 인덱스 → qnaList[current]를 읽어 질문 문장을 갱신 → 출력은 현재 질문 텍스트다.	test.html:43-86, 107-108
2	답변 버튼 A	입력은 선택된 보기의 type 값 → selectedTypes.push(opt.type) → 다음 질문으로 즉시 진행한다.	test.html:113-123
3	답변 버튼 B	처리 방식은 A와 동일하고 type 값만 다르다 → 화면 문장이 아닌 내부 type이 서버 집계 기준이 된다.	test.html:113-123
4	결과 이동 로직	입력은 selectedTypes, text, lat, lng → type1~type6 쿼리 문자열을 구성 → /test_result/ 로 이동한다. 반환값은 없고 브라우저 이동이 출력이다.	test.html:91-104

테스트 결과 화면: 태그 집계, GPT 추천, 가게 매칭, DB 저장을 한 번에 수행한다

현재 설명 기능과 계층: 추천 엔진 A > test_result_view() > 보조 모듈 > Recommendation 저장

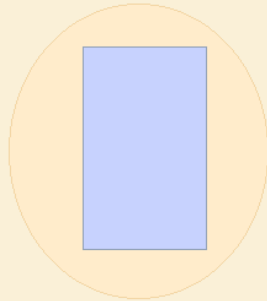
가장 핵심적인 백엔드 처리 구간이다 · 질문 답변은 score 딕셔너리로 집계되고, 그 score가 GPT 프롬프트 재료가 된다 · 성공과 실패 모두 Recommendation에 저장한다

테스트 결과 화면: 태그 집계 후 가게 추천과 저장 결과를 표시

설명용 예시 화면 · 실제 코드는 표의 위치 참조

당신에게 딱 맞는 음식은? **1**

현재 위치: 서울특별시 관악구 ... (예시 값)



오늘의 추천 가게

짬뽕지존 봉천점

매콤한 맛이 기분 전환에 딱이에요! (예시 설명)

입력 텍스트, 테스트 결과 type 목록, 카테고리, 키워드, 로고가 함께 노출된다.

2

DEBUG
입력 텍스트
위치
기분 태그
태그 점수
추천 가게
설명
카테고리
키워드

4

3

예시 값과 재구성 화면은 설명용이며, 실제 연결 판단은 다음 페이지 표의 코드 위치를 기준으로 했다.

테스트 결과 화면: 태그 집계, GPT 추천, 가게 매칭, DB 저장을 한 번에 수행한다 · 설명 표

현재 설명 기능과 계층: 추천 엔진 A > test_result_view() > 보조 모듈 > Recommendation 저장

번호	화면 요소와 쉬운 역할	입력·처리·출력	실제 코드 위치
1	추천 가게명과 설명	입력은 text, type1~type6, lat, lng → score 집계 후 create_yogiyo_prompt_with_options()와 GPT 호출 결과를 result로 만든다 → result.store, result.description이 화면에 출력된다.	views.py:392-433, 498-507 / test_result.html:45-56
2	보조 정보 영역	result.category, result.keywords와 text, types가 함께 표시된다 → 사용자는 왜 이 가게가 나왔는지 근거를 본다.	test_result.html:50-56
3	숨김 restaurantData	입력은 matched_restaurants 리스트 → mark_safe(json.dumps(...))로 직렬화 → 화면 재추천용 자바스크립트 입력으로 내려간다.	views.py:442-447, 498-500 / test_result.html:81-83, 90-113
4	디버깅 패널	DEBUG=True일 때만 노출 → score, 위치, 추천 결과를 개발 검수용으로 다시 보여준다.	test_result.html:61-75 / settings.py:26, 50

핵심 보조 함수: extract_keywords_from_store_name()는 가게명에서 명사를 뽑고(views.py:79-82), match_gpt_result_with_yogiyo()는 GPT가 말한 가게명과 요기요 실제 가게를 맞춘다(match_gpt_result_with_yogiyo.py:2-34).

직접 추천 결과 화면: 자유 입력을 먼저 분류한 뒤 한 번에 추천한다

현재 설명 기능과 계층: 추천 엔진 B > recommend_result() > classify_user_input()

테스트를 거치지 않는 빠른 추천 경로다 · 자유 입력의 의미를 기분·상황·기능·음식 중 하나로 먼저 분류한다

직접 추천 결과 화면: 자유 입력 분류 후 한 번에 추천

설명용 예시 화면 · 실제 코드는 표의 위치 참조

1 오늘의 추천 가게

마라탕전문점

매운 음식이 당길 때 바로 만족감을 주는 선택이에요! (예시 설명)

3 내가 검색한 단어: 매운 음식 추천해줘 (예시 값)

카테고리: 중식 · 관련 키워드: 마라, 얼얼함

4 평점: 4.9 · 주소: 서울시 관악구 ...

2



가게 로고

표시값은 result +
restaurants[0]을
함께 사용한다.

5

예시 값과 재구성 화면은 설명용이며, 실제 연결 판단은 다음 페이지 표의 코드 위치를 기준으로 했다.

직접 추천 결과 화면: 자유 입력을 먼저 분류한 뒤 한 번에 추천한다 · 설명 표

현재 설명 기능과 계층: 추천 엔진 B > recommend_result() > classify_user_input()

번호	화면 요소와 쉬운 역할	입력·처리·출력	실제 코드 위치
1	추천 가게명	입력은 text, lat, lng → classify_user_input(text)로 입력 유형을 먼저 얻고 → 해당 유형에 맞는 프롬프트로 GPT 추천을 만든다.	views.py:512-549 / classify_user_input.py:10-29
2	추천 이유와 로고	GPT가 준 description과 best_match의 logo_url을 함께 사용한다 → 문장과 시각 정보가 서로 다른 데이터 원천에서 온다.	views.py:556-569, 620-624 / recommend_result.html:34-47
3	검색한 단어 표시	입력은 request.GET.text → 화면에서는 사용자가 어떤 말로 검색했는지 재확인하는 용도다.	recommend_result.html:42
4	카테고리·키워드·평점·주소	카테고리와 키워드는 result 기준, 평점과 주소는 restaurants[0] 기준이다 → 조회 키와 저장 데이터의 역할이 다르다.	recommend_result.html:43-46 / views.py:559-569, 620-622
5	출력 조합 메모	result는 GPT 응답 중심, restaurants는 요기요 실가게 정보 중심 → 한 화면에서 두 데이터를 합쳐 보여준다.	views.py:549-569, 620-623

가게 목록 화면: 좌표별 가게 조회와 주소 표시만 담당한다

현재 설명 기능과 계층: 조회 전용 흐름 > restaurant_list_view() > Async HTTP + Cache

이 화면은 추천 저장 없이 조회 결과만 보여준다 · 같은 좌표는 5분 동안 캐시에서 재사용한다

가게 목록 화면: 위치별 배달 가능 가게 목록 조회

설명용 예시 화면 · 실제 코드는 표의 위치 참조

근처 배달 가능한 가게 목록

현재 위치: 서울특별시 관악구 ... (예시 값)



짬뽕지존 (☑ 4.8)

카테고리: 중식, 배달비: 2,000원, 리뷰 수: 120



샐러드하우스 (☑ 4.6)

카테고리: 중식, 배달비: 2,000원, 리뷰 수: 120



버거랩 (☑ 4.4)

카테고리: 중식, 배달비: 2,000원, 리뷰 수: 120

예시 값과 재구성 화면은 설명용이며, 실제 연결 판단은 다음 페이지 표의 코드 위치를 기준으로 했다.

가게 목록 화면: 좌표별 가게 조회와 주소 표시만 담당한다 · 설명 표

현재 설명 기능과 계층: 조회 전용 흐름 > restaurant_list_view() > Async HTTP + Cache

번호	화면 요소와 쉬운 역할	입력·처리·출력	실제 코드 위치
1	목록 제목	입력 없음 → 사용자가 현재 기능이 추천이 아닌 가게 조회임을 즉시 알게 한다.	restaurant_list.html:14
2	현재 위치 주소	입력은 lat, lng 또는 IP 기반 loc → Kakao Maps JavaScript geocoder가 좌표를 주소로 바꿔 표시한다.	restaurant_list.html:17-19, 74-129
3	가게 카드 반복 영역	입력은 restaurants 배열 → 이름, 평점, 카테고리, 배달비, 리뷰 수를 for 루프로 출력한다.	restaurant_list.html:20-32
4	백엔드 조회 처리	입력은 query string lat/lng → cache_key=restaurants:{lat}:{lng}를 먼저 조회 → 없으면 fetch_yogiyo_data()가 비동기 HTTP로 요기요 목록을 가져온다.	views.py:269-287, 291-325

데이터베이스 구조: Recommendation 한 테이블에 추천 이력을 저장한다

현재 설명 기능과 계층: DB > models.py > migration > admin > settings

앱 전용 저장 구조는 Recommendation 한 테이블이다 · 테스트 기반 추천과 직접 추천이 같은 테이블을 공유한다 · 조회용 좌표와 저장용 매칭 키는 다른 필드에 분리된다

데이터베이스 구조: 추천 이력 1개 테이블과 주변 설정

설명용 예시 화면 · 실제 코드는 표의 위치 참조

Recommendation 테이블

- id: 기본 키
- input_text: 사용자 자유 입력
- selected_types: 테스트 집계 또는 입력 분류
- recommended_store: 최종 추천 가게명
- description: 추천 이유 문장
- category: 대표 카테고리
- keywords: 화면 표시용 키워드 배열
- latitude / longitude: 조회 좌표
- user_ip: 요청 IP
- is_success: 추천 성공 여부
- gpt_raw_response: 원본 응답 보관
- matched_restaurant_id: 요기요 가게 ID
- created_at: 저장 시각

런타임 설정
DB에 저장한다. PostgreSQL 연결값을 기본
(ENGINE, NAME, USER, PASSWORD,
HOST, PORT)

마이그레이션
0001initial.py: Recommendation 생성

관계 메모
관계용 테이블과 외래키를 기록한다.
관리 화면은 admin.py에서

예시 값과 재구성 화면은 설명용이며, 실제 연결 판단은 다음 페이지 표의 코드 위치를 기준으로 했다.

데이터베이스 구조: Recommendation 한 테이블에 추천 이력을 저장한다 · 설명 표

현재 설명 기능과 계층: DB > models.py > migration > admin > settings

번호	화면 요소와 쉬운 역할	입력·처리·출력	실제 코드 위치
1	input_text	의미는 사용자의 원문 입력이다. 테스트형 흐름에서는 text, 직접 추천 흐름에서도 text로 들어와 추천 맥락과 감사 이력의 기준이 된다.	models.py:4, migrations/0001_initial.py:18
2	selected_types	테스트형 흐름에서는 score 딕셔너리, 직접 추천 흐름에서는 {input_category: 분류값}이 저장된다. 같은 필드지만 저장 의미가 경로별로 다르다.	models.py:5, views.py:451-452, 574
3	matched_restaurant_id	화면 조회 시 쓰는 lat/lng와 다르게, 이 값은 최종 추천을 실제 요기요 가게 한 건과 연결하는 저장 키 역할이다.	models.py:15, views.py:462, 584
4	성공/실패 기록	is_success와 gpt_raw_response로 추천 품질과 재현 정보를 남긴다. 예외가 나도 fallback 결과와 함께 저장된다.	models.py:13-14, views.py:449-496, 571-617
5	관리 화면과 관계	admin.py는 Recommendation만 등록한다. 앱 전용 Foreign Key 관계는 없고, Django 기본 auth/session 테이블과 느슨하게 공존한다.	admin.py:4-8
6	운영 DB 메모	settings.py는 PostgreSQL 연결을 기본으로 둔다. 반면 저장소의 db.sqlite3 스냅샷에서는 Recommendation 테이블이 확인되지 않아, 로컬 파일 상태와 운영 설정을 구분해 인수인계해야 한다.	settings.py:106-115, migrations/0001_initial.py:14-31